

Learning C++ from the inventor of C++ – Exercise 2 (Measurement)

Katrine Sandø (202502268)

2026-07-29

What was the results of your measurements? Is this surprising?

Benchmark	Median	Spread (ms)	Elements scanned	ns/element
find 7 in vector<int>, hit at middle	29.6 ms	29.3 – 33.6	50 000 000	0.59
find 7 in vector<int>, miss	70.7 ms	69.6 – 72.0	100 000 000	0.71
find_if $x < 7$ in vector<int>, hit at middle	28.6 ms	28.0 – 33.2	50 000 000	0.57
find_if $x < 7$ in vector<int>, miss	58.0 ms	57.6 – 64.1	100 000 000	0.58
find string in vector<string>, miss	8.2 ms	8.2 – 8.3	1 000 000	8.17
find string in vector<string>, hit at middle	4.1 ms	4.0 – 7.9	500 000	8.10

Table 1: Results from measurements

Benchmark Runtime Variance Across Runs

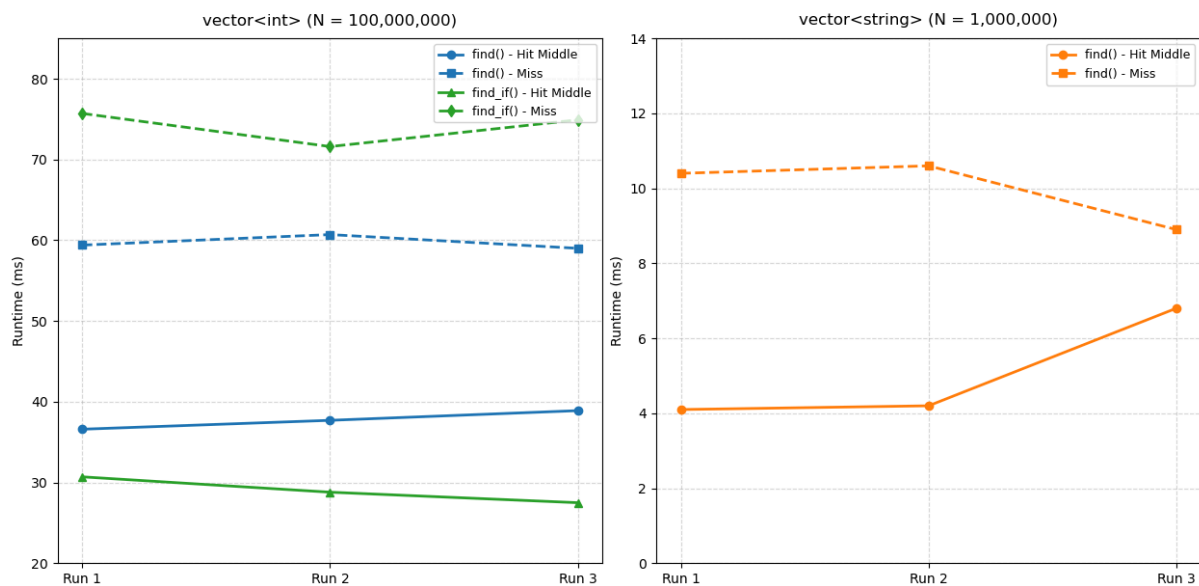


Figure 1: Run times for tests cases

It makes sense to me that the miss cases were a lot slower, since `find()` had to iterate through the whole vector instead of terminating as soon as it hit the target.

I originally thought `vector<string>` was faster, because its total runtime was only around 10 ms compared to nearly 60 ms for `vector<int>`. But I realized the `vector<int>` had 100,000,000 elements while the `vector<string>` only had 1,000,000.

Looking at the cost per element scanned, `vector<int>` was actually way faster (≈ 0.59 ns vs ≈ 10.4 ns per element). This makes sense because integers are 4 bytes and sit tightly in the CPU cache, whereas strings take up a lot more memory per element and require comparing individual characters.

I expected the time scaling to be strictly linear, where scanning 100M elements would take double the time of scanning 50M elements (around 75 ms instead of 59.4 ms). The full scan ended up being slightly faster per element than the half scan, likely because the CPU's memory pre-fetcher gets more efficient during long sequential reads across continuous memory.